Lesson 1.8: EDA Basic

Welcome to the Exploratory Data Analysis (EDA) lesson! This notebook is designed to guide you through the essential steps of taking a raw dataset, understanding its structure, cleaning it, and transforming it for analysis.

Structure:

- **Section 1: The First Look:** How to read files and get a quick statistical summary.
- **Section 2: The Cleanup:** Handling the messy reality of data (missing values, duplicates, outliers).
- **Section 3: Refinement & Transformation:** preparing data for machine learning or deep analysis (mapping, string manipulation).

For Learners: Read the comments in the code to understand exactly what each parameter does.

Setup

As always, we start by importing the necessary libraries. `pandas` is our primary tool for data manipulation, and `numpy` helps with numerical operations.

In [1]:
```
import pandas as pd
import numpy as np
```

Section 1: The First Look: Inspection & Summary

Goal: Load data from different file formats and get a "health check" of the data using descriptive statistics.

We will cover:

1. Reading CSV and Text files (handling headers, delimiters, and special characters).
2. Summarizing data using `.describe()`, `.sum()`, and `.value_counts()`.

Part 1: Reading Data

Pandas is incredibly flexible with input formats. Let's start by reading a standard Comma-Separated Values (CSV) file.

In [2]:
```
# Let's inspect the raw file content first using a shell command
!cat ../data/ex1.csv
```

```
a,b,c,d,message
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

In [3]:
```
# Use read_csv to load the data into a DataFrame
# By default, it assumes the first row is the header
df = pd.read_csv("../data/ex1.csv")
```

```
df
```

Out[3]:

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Scenario: What if the file doesn't have a header row? If we don't specify this, pandas will mistakenly use the first row of data as column names.

In [4]:
```
!cat ../data/ex2.csv
```

```
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

In [6]:
```
# Option 1: Tell pandas there is no header. It will assign integers (0, 1, 2...) as c
# pd.read_csv("../data/ex2.csv")
pd.read_csv("../data/ex2.csv", header=None)
```

Out[6]:

	0	1	2	3	4
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

In [7]:
```
# Option 2: Provide your own column names using the 'names' parameter
pd.read_csv("../data/ex2.csv", names=["a", "b", "c", "d", "message"])
```

Out[7]:

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Indexing: You can also designate a specific column to be the index (row labels) of the DataFrame, rather than the default 0, 1, 2... index.

In [8]:
```
names = ["a", "b", "c", "d", "message"]

# Use 'index_col' to set the 'message' column as the index
pd.read_csv("../data/ex2.csv", names=names, index_col="message")
```

Out[8]:

	a	b	c	d
message				
hello	1	2	3	4
world	5	6	7	8
foo	9	10	11	12

In [10]:
```
custom_idx_df = pd.read_csv("../data/ex2.csv", names=names, index_col="message")

# Select row where the message is "hello"
```

```
# custom_idx_df.loc["hello"]

# vs using integer position
custom_idx_df.iloc[0]
```

Out[10]:
```
a    1
b    2
c    3
d    4
Name: hello, dtype: int64
```

Irregular Separators: Sometimes data isn't separated by commas. It might be tabs, spaces, or a variable amount of whitespace. We can use Regular Expressions (Regex) to handle this.

In [11]:
```
# Inspect a file with messy whitespace
!cat ../data/ex3.txt
```

```
A         B         C
aaa -0.264438 -1.026059 -0.619500
bbb  0.927272  0.302904 -0.032399
ccc -0.264273 -0.386314 -0.217601
ddd -0.871858 -0.348382  1.100491
```

In [ ]:
```
# sep="\s+" is a regex that means "one or more whitespace characters"
# "r" prefix means it's a raw string, so we don't have to escape the backslash
result = pd.read_csv("../data/ex3.txt", sep=r"\s+")

result
```

Out[ ]:

	A	B	C
aaa	-0.264438	-1.026059	-0.619500
bbb	0.927272	0.302904	-0.032399
ccc	-0.264273	-0.386314	-0.217601
ddd	-0.871858	-0.348382	1.100491

Skipping Rows: Sometimes files contain comments or metadata at the top that we want to ignore.

In [14]:
```
!cat ../data/ex4.csv
```

```
# hey!
a,b,c,d,message
# just wanted to make things more difficult for you
# who reads CSV files with computers, anyway?
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

In [15]:
```
# Skip specific rows by index (0, 2, and 3 here) to get to the clean data
pd.read_csv("../data/ex4.csv", skiprows=[0, 2, 3])
```

Out[15]:

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Handling Missing Values (at load time): Pandas is smart about identifying missing data (empty strings, 'NA', 'NULL'), but we can also define our own "sentinels" for missing values.

In [16]:
```
!cat ../data/ex5.csv
```

```
something,a,b,c,d,message
one,1,2,3,4,NA
two,5,6,,8,world
three,9,10,11,12,NULL
```

In [17]:
```
# Default behavior: reads 'NA' and 'NULL' as NaN (Not a Number/Missing)
result = pd.read_csv("../data/ex5.csv")

result
```

Out[17]:

	something	a	b	c	d	message
0	one	1	2	3.0	4	NaN
1	two	5	6	NaN	8	world
2	three	9	10	11.0	12	NaN

In [ ]:
```
# Verify where the missing values are detected
result.isna()

# legacy method, also works
# result.isnull()
```

Out[ ]:

	something	a	b	c	d	message
0	False	False	False	False	False	True
1	False	False	False	True	False	False
2	False	False	False	False	False	True

In [20]:
```
# For large tables, not practical to display fully, so we can count the number of mis
# result.isna().sum()

# Check if any column has missing values
result.isna().any()
```

Out[20]:
```
something    False
a            False
b            False
c             True
d            False
message       True
dtype: bool
```

In [23]:
```
# Testing for missing values
from io import StringIO

# Create a string that simulates a CSV file
products_csv = """
product_id,product_name,price,remarks
1,Apple iPhone,1899,
2,Apple iPad,999,NA
3,Apple Watch,599,NULL
4,Apple Macbook,599,
5,Apple AirPods,199,na
6,Apple TV,399, NULL
"""

products_df = pd.read_csv(StringIO(products_csv))
products_df
```

Out[23]:

	product_id	product_name	price	remarks
0	1	Apple iPhone	1899	NaN
1	2	Apple iPad	999	NaN
2	3	Apple Watch	599	NaN
3	4	Apple Macbook	599	
4	5	Apple AirPods	199	na
5	6	Apple TV	399	NULL

You can customize what is considered "Missing" for each column specifically.

In [25]:
```
!cat ../data/ex5.csv
```

```
something,a,b,c,d,message
one,1,2,3,4,NA
two,5,6,,8,world
three,9,10,11,12,NULL
```

In [ ]:
```
# Define a dictionary of sentinels
# For column 'message', treat 'NULL' and 'NA' as null
# For column 'something', treat 'two' as null
sentinels = {"message": ["NULL", "NA"], "something": ["two"]}

# keep_default_na=False means we don't want to treat the default sentinels as null, o
pd.read_csv("../data/ex5.csv", na_values=sentinels, keep_default_na=False)

# Why need to customize sentinels?
# - some datasets may use "missing", "n/a" or even specifc codes -999 to indicate mis
# - NA could mean North America, Not Applicable, Not Available
```

Out[ ]:

	something	a	b	c	d	message
0	one	1	2	3	4	NaN
1	NaN	5	6		8	world
2	three	9	10	11	12	NaN

Reading Excel: Reading Excel files works similarly, but we can specify sheet names.

In [ ]:
```
# Load the Excel file object
# xlsx = pd.ExcelFile("../data/Resaleflatpricesbasedonregistrationdate.xlsx", engine=
xlsx = pd.ExcelFile("../data/Resaleflatpricesbasedonregistrationdate.xlsx")

# List available sheets
xlsx.sheet_names
```

Out[ ]:
```
['2017', '2018', '2019']
```

In [ ]:
```
print("File path", xlsx.io)
print("Engine", xlsx.engine)
# engine is the underlying library used to read the Excel file, such as 'openpyxl' or
```

```
File path ../data/Resaleflatpricesbasedonregistrationdate.xlsx
Engine openpyxl
```

In [32]:
```
!pip list | grep openpyxl
```

```
openpyxl                    3.0.10
```

In [ ]:
```
# Parse a specific sheet into a DataFrame
xlsx.parse(sheet_name="2017")
```

Out[ ]:

	month	town	flat_type	block	street_name	storey_range	floor_area_sqm	flat_mode
0	2017-01	ANG MO KIO	2 ROOM	406	ANG MO KIO AVE 10	10 TO 12	44	Improvec
1	2017-01	ANG MO KIO	3 ROOM	108	ANG MO KIO AVE 4	01 TO 03	67	Nev Generatior
2	2017-01	ANG MO KIO	3 ROOM	602	ANG MO KIO AVE 5	01 TO 03	67	Nev Generatior
3	2017-01	ANG MO KIO	3 ROOM	465	ANG MO KIO AVE 10	04 TO 06	68	Nev Generatior
4	2017-01	ANG MO KIO	3 ROOM	601	ANG MO KIO AVE 5	01 TO 03	67	Nev Generatior
...	...	...	...	...	...	...	...	..
1180	2017-01	YISHUN	EXECUTIVE	387	YISHUN RING RD	07 TO 09	142	Apartmen
1181	2017-01	YISHUN	EXECUTIVE	877	YISHUN ST 81	04 TO 06	142	Apartmen
1182	2017-01	YISHUN	EXECUTIVE	836	YISHUN ST 81	04 TO 06	146	Maisonette
1183	2017-01	YISHUN	EXECUTIVE	633A	YISHUN ST 61	01 TO 03	164	Apartmen
1184	2017-01	YISHUN	EXECUTIVE	709	YISHUN AVE 5	07 TO 09	163	Adjoinec fla

1185 rows × 11 columns

In [38]:
```
# Shortcut: Read directly without creating an ExcelFile object first
frame = pd.read_excel("../data/Resaleflatpricesbasedonregistrationdate.xlsx", sheet_n

frame
```

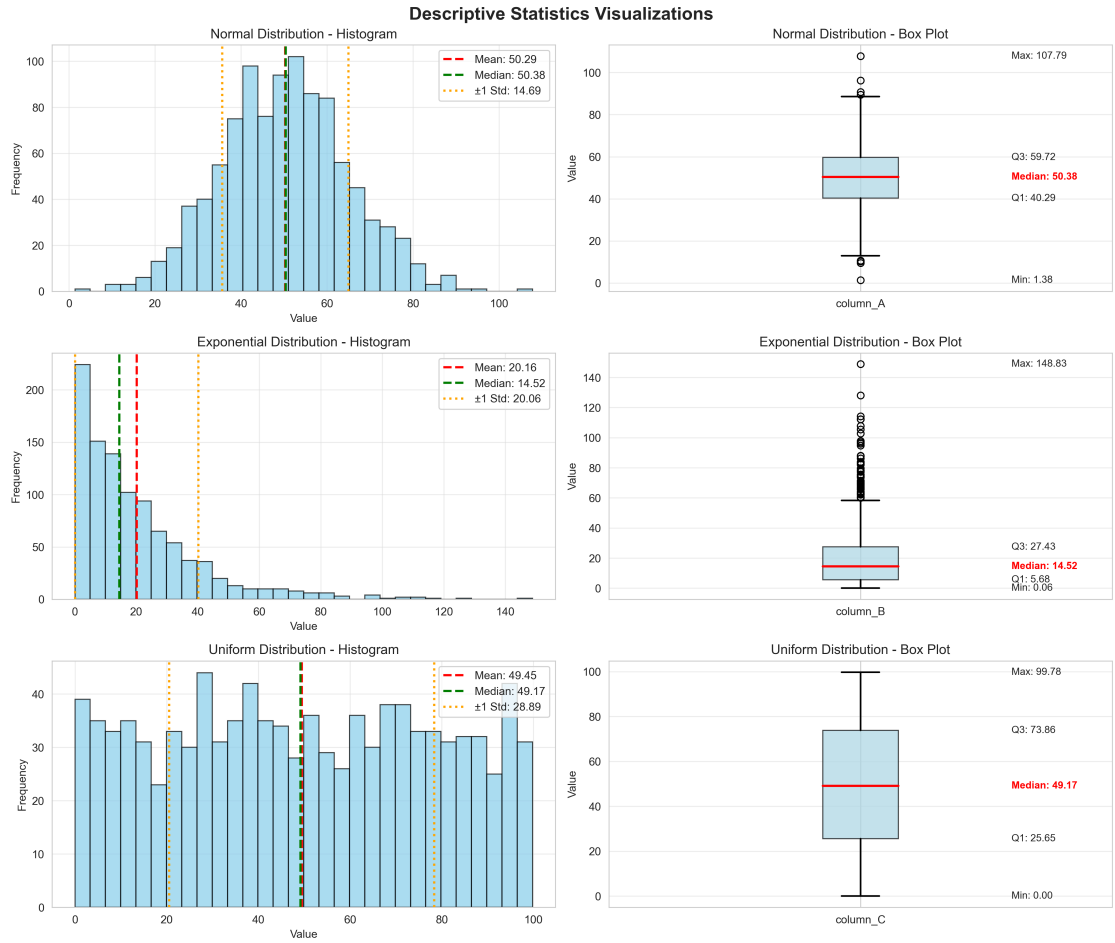
Out[38]:

	month	town	flat_type
0	2017-01	ANG MO KIO	2 ROOM
1	2017-01	ANG MO KIO	3 ROOM
2	2017-01	ANG MO KIO	3 ROOM
3	2017-01	ANG MO KIO	3 ROOM
4	2017-01	ANG MO KIO	3 ROOM

Part 2: Summarizing and Computing Descriptive Statistics

Once data is loaded, we need to understand its "shape". We use summary statistics to get a birds-eye view.

📊 **Visual Illustration Available**: See



for visual representations of descriptive statistics including histograms, box plots, and how statistics like mean, median, and quartiles relate to data distributions.

Let's create a sample DataFrame with some missing values to test our summary methods.

In [ ]:
```
# Create a df with 4 rows and 2 columns, where some values are missing (NaN)
df = pd.DataFrame([
  [1.4, np.nan],
  [7.1, -4.5],
  [np.nan, np.nan],
  [0.75, -1.3]],
  index=["a", "b", "c", "d"],
  columns=["one", "two"]
  )

df
```

Out[ ]:

	one	two
a	1.40	NaN
b	7.10	-4.5
c	NaN	NaN
d	0.75	-1.3

In [ ]:
```
# Why use np.nan instead of None for missing values in numeric arrays?
arr1 = np.array([1.0,2.0, None])
print(arr1)
print(arr1.dtype)

arr2 = np.array([1.0, 2.0, np.nan])
```

```
print(arr2)
print(arr2.dtype)

# print(arr1 * 2)
print(arr2 * 2)
```

```
[1.0 2.0 None]
object
[ 1.  2. nan]
float64
[ 2.  4. nan]
```

In [49]:
```
s1 = pd.Series([1.0, 2.0, None]) # Pandas converts None to NaN in numeric Series
s2 = pd.Series([1.0, 2.0, np.nan])
print(s1)
print(s2)
```

```
0    1.0
1    2.0
2    NaN
dtype: float64
0    1.0
1    2.0
2    NaN
dtype: float64
```

Reductions: Methods like `.sum()` or `.mean()` reduce a Series of numbers to a single value. By default, they operate down the rows (`axis=0`).

In [50]:
```
df
```

Out[50]:

	one	two
a	1.40	NaN
b	7.10	-4.5
c	NaN	NaN
d	0.75	-1.3

In [ ]:
```
# Sums down the rows (returns sum for each column)
df.sum()
# df.sum(axis=0)
# df.sum(axis="index")
```

Out[ ]:
```
one    9.25
two   -5.80
dtype: float64
```

In [52]:
```
# Sums across the columns (returns sum for each row)
# axis=1 is synonymous with axis='columns'
df.sum(axis="columns")
```

Out[52]:
```
a    1.40
b    2.60
c    0.00
d   -0.55
dtype: float64
```

Handling NAs in Calculations: By default, pandas ignores NaNs (treats them as zero for sums). We can change this with `skipna`.

In [53]:
```
# If any value is NaN, the result is NaN
# by default, skipna=True, so it ignores NaN and returns the sum of the non-NaN value
```

```
# when use skipna=False? e.g. flag if there are NaN values, should not sum e.g. accou
df.sum(skipna=False)
```

Out[53]:
```
one    NaN
two    NaN
dtype: float64
```

In [54]:
```
df.sum(axis=1, skipna=False)
```

Out[54]:
```
a     NaN
b    2.60
c     NaN
d   -0.55
dtype: float64
```

Indirect Statistics: Finding *where* the max or min value is located (the index label).

In [57]:
```
df
```

Out[57]:

	one	two
a	1.40	NaN
b	7.10	-4.5
c	NaN	NaN
d	0.75	-1.3

In [55]:
```
# Returns the index of the maximum value
df.idxmax()
```

Out[55]:
```
one    b
two    d
dtype: object
```

In [56]:
```
# Returns the index of the minimum value
df.idxmin()
```

Out[56]:
```
one    d
two    b
dtype: object
```

In [ ]:
```
sample_df = pd.DataFrame({
  "name": ["Alice", "Bob", "SCTP Study Buddy", "Einstein"],
  "age": [25, 30, 1, 40],
  "score": [85, 76, 64, 99]
})

sample_df
```

Out[ ]:

	name	age	score
0	Alice	25	85
1	Bob	30	76
2	SCTP Study Buddy	1	64
3	Einstein	40	99

In [ ]:
```
# Find index of max value in each column
sample_df[["age", "score"]].idxmax()
# # sample_df["age"].idxmax()
```

```
# this selects all columns, including non-numeric, which will result in an error
sample_df.idxmax()
```

Out[ ]:
```
age      3
score    3
dtype: int64
```

In [ ]:
```
print("Row with minimum age:")
# sample_df["age"].idxmin()
# sample_df.loc[sample_df["age"].idxmin()] gets the index of the row with the minimum
print(sample_df.loc[sample_df["age"].idxmin()])
print()

print("Row with maximum score:")
# sample_df["score"].idxmax() gets the index of the row with the maximum score, then
print(sample_df.loc[sample_df["score"].idxmax()])
```

```
Row with minimum age:
name     SCTP Study Buddy
age                     1
score                  64
Name: 2, dtype: object

Row with maximum score:
name     Einstein
age            40
score          99
Name: 3, dtype: object
```

Accumulations: Computing cumulative sums.

In [63]:
```
df.cumsum()
```

Out[63]:

	one	two
a	1.40	NaN
b	8.50	-4.5
c	NaN	NaN
d	9.25	-5.8

The `.describe()` method: This is your best friend for a quick snapshot of numeric data.

In [64]:
```
# Provides count, mean, std, min, quartiles, and max
df.describe()
```

Out[64]:

	one	two
count	3.000000	2.000000
mean	3.083333	-2.900000
std	3.493685	2.262742
min	0.750000	-4.500000
25%	1.075000	-3.700000
50%	1.400000	-2.900000
75%	4.250000	-2.100000
max	7.100000	-1.300000

Categorical/Non-Numeric Data: `.describe()` behaves differently for string data, showing counts and uniqueness.

In [65]:
```
obj = pd.Series(["c", "a", "d", "a", "b", "b", "c", "c"])

# For object data, we get count, unique, top (most frequent), and freq
obj.describe()
```

Out[65]:
```
count     8
unique    4
top       c
freq      3
dtype: object
```

In [66]:
```
sample_df.describe()
```

Out[66]:

	age	score
count	4.000000	4.000000
mean	24.000000	81.000000
std	16.552945	14.764823
min	1.000000	64.000000
25%	19.000000	73.000000
50%	27.500000	80.500000
75%	32.500000	88.500000
max	40.000000	99.000000

In [67]:
```
sample_df.describe(include="all")
```

Out[67]:

	name	age	score
count	4	4.000000	4.000000
unique	4	NaN	NaN
top	Alice	NaN	NaN
freq	1	NaN	NaN
mean	NaN	24.000000	81.000000
std	NaN	16.552945	14.764823
min	NaN	1.000000	64.000000
25%	NaN	19.000000	73.000000
50%	NaN	27.500000	80.500000
75%	NaN	32.500000	88.500000
max	NaN	40.000000	99.000000

In [68]:
```
# Get unique values
obj.unique()
```

Out[68]:
```
array(['c', 'a', 'd', 'b'], dtype=object)
```

In [69]:
```
# Get frequency counts of each unique value
obj.value_counts()
```

Out[69]:
```
c    3
a    2
b    2
d    1
dtype: int64
```

🛠️ **Exercise:** Sort value counts in ascending order.

In [ ]:
```python
obj.value_counts().sort_index()
# sort_index sorts the resulting Series in ascending order
```

Out[ ]:
```
a    2
b    2
c    3
d    1
dtype: int64
```

Section 2: The Cleanup: Missing Data, Duplicates & Outliers

Goal: Real-world data is dirty. In this section, we will learn techniques to identify and fix common quality issues.

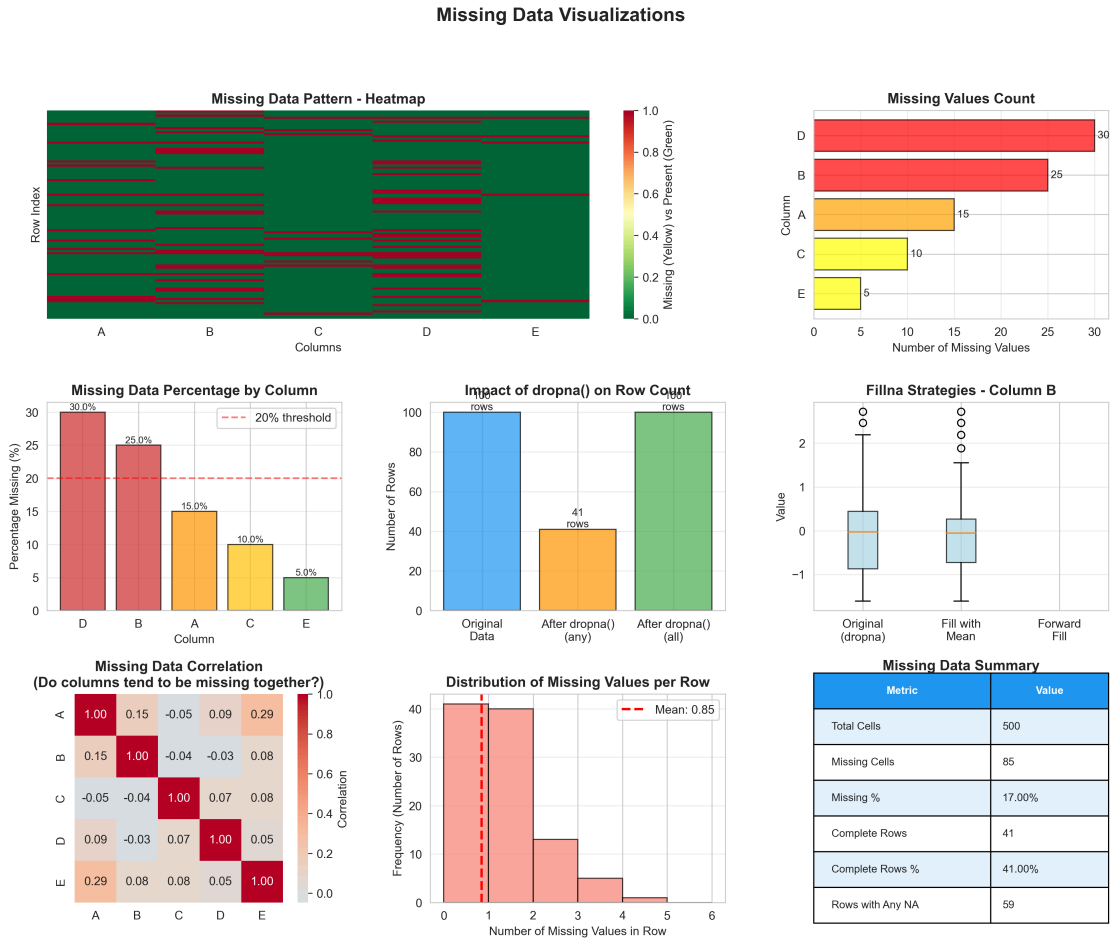
We will cover:

1. **Missing Data:** Detecting nulls and deciding whether to drop them or fill them.
2. **Duplicates:** Finding and removing repeated records.
3. **Outliers:** identifying extreme values using boolean indexing.

Part 1: Handling Missing Data

Missing data is often represented as `NaN` (Not a Number) or `None`.

📊 **Visual Illustration Available**: See



for visualizations of missing data patterns, heatmaps, and comparison of different strategies for handling missing values.

In [71]:
```
float_data = pd.Series([1.2, -3.5, np.nan, 0])

float_data
```

Out[71]:
```
0    1.2
1   -3.5
2    NaN
3    0.0
dtype: float64
```

In [72]:
```
# .isna() returns a boolean mask (True if missing, False if present)
float_data.isna()
```

Out[72]:
```
0    False
1    False
2     True
3    False
dtype: bool
```

The built-in Python `None` value is also treated as NA in pandas object arrays.

In [73]:
```
string_data = pd.Series(["aardvark", np.nan, None, "avocado"])

string_data
```

Out[73]:
```
0    aardvark
1         NaN
2        None
3     avocado
dtype: object
```

In [74]:
```
string_data.isna()
```

Out[74]:
```
0    False
1     True
2     True
3    False
dtype: bool
```

Strategy 1: Dropping Missing Data (`dropna`)

The simplest strategy is to just remove the rows or columns that contain missing values.

In [75]:
```
data = pd.Series([1, np.nan, 3.5, np.nan, 7])

# Removes all NaN values from the Series
data.dropna()
```

Out[75]:
```
0    1.0
2    3.5
4    7.0
dtype: float64
```

In [76]:
```
# This is equivalent to boolean filtering:
data[data.notna()]
```

Out[76]:
```
0    1.0
2    3.5
4    7.0
dtype: float64
```

With DataFrames, `dropna` by default drops **any row** containing **any missing value**.

In [77]:
```
data = pd.DataFrame([[1., 6.5, 3.], [1., np.nan, np.nan],
                     [np.nan, np.nan, np.nan], [np.nan, 6.5, 3.]])

data
```

Out[77]:

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
2	NaN	NaN	NaN
3	NaN	6.5	3.0

In [78]:
```
# Drops rows 1, 2, and 3 because they have at least one NaN
data.dropna()
```

Out[78]:

	0	1	2
0	1.0	6.5	3.0

We can control this behavior. `how='all'` only drops rows where **all** values are NaN.

In [79]:
```
data.dropna(how="all")
```

Out[79]:

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
3	NaN	6.5	3.0

To drop **columns** instead of rows, pass `axis=1`.

In [80]:

```
# Let's add a column of all NaNs first
data[4] = np.nan

data
```

Out[80]:

	0	1	2	4
0	1.0	6.5	3.0	NaN
1	1.0	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	6.5	3.0	NaN

In [81]:

```
# Drops the column '4' because it is all NaNs
data.dropna(axis="columns", how="all")
```

Out[81]:

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
2	NaN	NaN	NaN
3	NaN	6.5	3.0

We can also set a **threshold**: keep only rows containing at least `n` valid observations.

In [82]:

```
df = pd.DataFrame(np.random.standard_normal((7, 3)))
# Set some missing values
df.iloc[:4, 1] = np.nan
df.iloc[:2, 2] = np.nan

df
```

Out[82]:

	0	1	2
0	-0.001293	NaN	NaN
1	1.775056	NaN	NaN
2	-1.107639	NaN	0.089078
3	-0.321476	NaN	-0.897529
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

In [83]:

```
df.dropna()
```

Out[83]:

	0	1	2
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

In [84]:

```
# Keep rows that have at least 2 non-NaN values
df.dropna(thresh=2)
```

Out[84]:

	0	1	2
2	-1.107639	NaN	0.089078
3	-0.321476	NaN	-0.897529
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

Strategy 2: Filling Missing Data (`fillna`)

Instead of losing data, we can fill the holes with a constant or a calculated value.

In [85]:

```
# Replace all NaNs with 0
df.fillna(0)
```

Out[85]:

	0	1	2
0	-0.001293	0.000000	0.000000
1	1.775056	0.000000	0.000000
2	-1.107639	0.000000	0.089078
3	-0.321476	0.000000	-0.897529
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

You can specify different fill values for each column:

In [86]:

```
# Fill column 1 with 0.5, and column 2 with 0
df.fillna({1: 0.5, 2: 0})
```

Out[86]:

	0	1	2
0	-0.001293	0.500000	0.000000
1	1.775056	0.500000	0.000000
2	-1.107639	0.500000	0.089078
3	-0.321476	0.500000	-0.897529
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

Forward/Backward Fill: Useful for time-series data, where you propagate the last valid observation forward or backward.

In [87]:
```
df
```

Out[87]:

	0	1	2
0	-0.001293	NaN	NaN
1	1.775056	NaN	NaN
2	-1.107639	NaN	0.089078
3	-0.321476	NaN	-0.897529
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

In [88]:
```
# Propagate next valid value backward to fill gaps
df.fillna(method="bfill")
```

Out[88]:

	0	1	2
0	-0.001293	0.649757	0.089078
1	1.775056	0.649757	0.089078
2	-1.107639	0.649757	0.089078
3	-0.321476	0.649757	-0.897529
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

In [89]:
```
# Same, but limit how many rows are filled consecutively
df.fillna(method="bfill", limit=2)
```

Out[89]:

	0	1	2
0	-0.001293	NaN	0.089078
1	1.775056	NaN	0.089078
2	-1.107639	0.649757	0.089078
3	-0.321476	0.649757	-0.897529
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

Imputation: Filling with the mean or median is a very common technique.

In [90]:
```
data = pd.Series([1., np.nan, 3.5, np.nan, 7])

data
```

Out[90]:
```
0    1.0
1    NaN
2    3.5
3    NaN
4    7.0
dtype: float64
```

In [91]:
```
# Fill with the mean of the available data
data.fillna(data.mean())
```

Out[91]:
```
0    1.000000
1    3.833333
2    3.500000
3    3.833333
4    7.000000
dtype: float64
```

🛠️ Exercise: Handling Missing Data

Practice using these methods on the dataframe below.

In [ ]:
```
df = pd.DataFrame(np.random.standard_normal((6, 3)))

df.iloc[[2,4,5], 1] = np.nan
df.iloc[4:, 2] = np.nan
df.iloc[3:5, 0] = np.nan

df
```

> **Tasks:**
>
> 1. Remove rows with *any* missing values.
> 2. Remove rows with *all* missing values.
> 3. Fill missing values with forward fill.
> 4. Fill missing values with mean of the column.

In [92]:
```
# > 1. Remove rows with *any* missing values.
df.dropna()
```

Out[92]:

	0	1	2
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

In [93]:
```
# > 2. Remove rows with *all* missing values.
df.dropna(how="all")
```

Out[93]:

	0	1	2
0	-0.001293	NaN	NaN
1	1.775056	NaN	NaN
2	-1.107639	NaN	0.089078
3	-0.321476	NaN	-0.897529
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

In [ ]:
```
# > 3. Fill missing values with forward fill.
df.fillna(method="ffill") # df.ffill()
```

In [94]:
```
# > 4. Fill missing values with mean of the column.
df.fillna(df.mean())
```

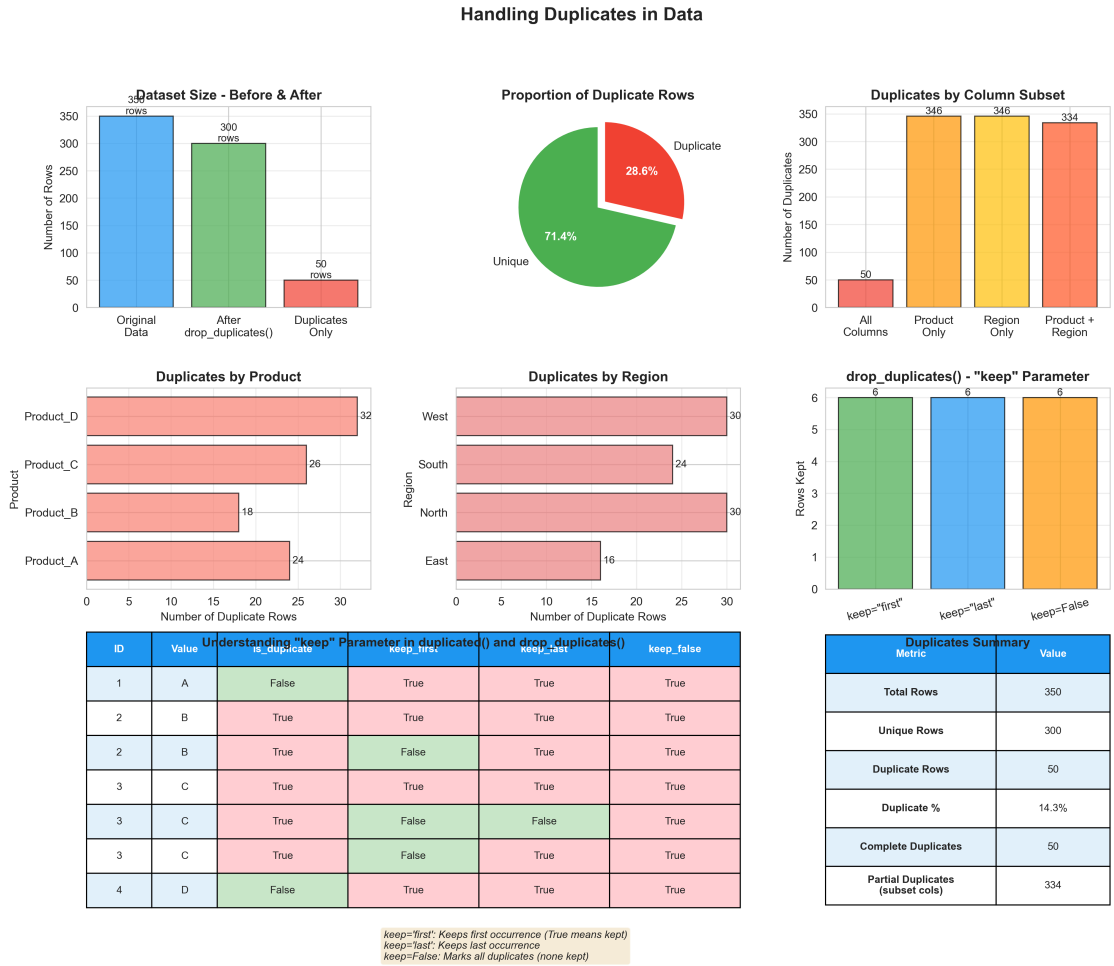
Out[94]:

	0	1	2
0	-0.001293	0.074802	-0.247958
1	1.775056	0.074802	-0.247958
2	-1.107639	0.074802	0.089078
3	-0.321476	0.074802	-0.897529
4	0.582525	0.649757	-0.250957
5	-0.296390	-0.894394	1.039655
6	-2.901564	0.469043	-1.220036

Part 2: Handling Duplicates

Duplicate rows can skew analysis and models. We typically identify them and remove them.

📊 **Visual Illustration Available**: See



for visualizations showing how duplicates affect datasets and the impact of different `keep` parameter options.

In [95]:
```
data = pd.DataFrame({"k1": ["one", "two"] * 3 + ["two"],
                     "k2": [1, 1, 2, 3, 3, 4, 4]})

data
```

Out[95]:

	k1	k2
0	one	1
1	two	1
2	one	2
3	two	3
4	one	3
5	two	4
6	two	4

`duplicated()` returns a boolean Series indicating whether each row has been seen before.

In [96]:
```
data.duplicated()
```

Out[96]:
```
0    False
1    False
2    False
3    False
4    False
5    False
6     True
dtype: bool
```

`drop_duplicates()` creates a new DataFrame with the duplicates removed.

In [97]:
```
data.drop_duplicates()
```

Out[97]:

	k1	k2
0	one	1
1	two	1
2	one	2
3	two	3
4	one	3
5	two	4

Subset: Sometimes we only care about duplicates in specific columns.

In [ ]:
```
# Add a column to make the rows unique
data["v1"] = range(7)

data
```

Out[ ]:

	k1	k2	v1
0	one	1	0
1	two	1	1
2	one	2	2
3	two	3	3
4	one	3	4
5	two	4	5
6	two	4	6

In [99]:
```
# Drop duplicates considering only column 'k1'
data.drop_duplicates(subset=["k1"])
```

Out[99]:

	k1	k2	v1
0	one	1	0
1	two	1	1

Keep: By default, it keeps the first occurrence. We can keep the last one instead.

In [ ]:
```
data.drop_duplicates(subset=["k1", "k2"], keep=False)

# keep="first" (default) keeps the first occurrence and drops the rest
```

```
# keep="last" keeps the last occurrence and drops the rest
# keep=False drops all duplicates, so only unique rows are kept
```

Out[ ]:

	k1	k2	v1
0	one	1	0
1	two	1	1
2	one	2	2
3	two	3	3
4	one	3	4
5	two	4	5

Part 3: Handling Outliers

Outliers are extreme values that deviate significantly from the rest of the data. We can filter them using boolean indexing.

> 📊 **Visual Illustration Available**: See
>
>
>
>
> **Statistical Summary Comparison**
>
> | Metric | Original | Outliers Removed | Capped/Winsorized |
> |---|---|---|---|
> | Total Data Points | 206 | 197 | 206 |
> | Mean | 99.33 | 99.18 | 99.33 |
> | Std Dev | 16.93 | 13.21 | 15.07 |
> | Min | 35.00 | 70.19 | 61.36 |
> | Max | 160.00 | 132.86 | 135.66 |
> | Q1 (25%) | 89.22 | 89.47 | 89.22 |
> | Median (50%) | 99.94 | 99.80 | 99.94 |
> | Q3 (75%) | 107.80 | 107.24 | 107.80 |
> | Outliers Detected (IQR) | 9 | N/A | 0 (capped) |
>
> for visualizations of outlier detection methods (IQR, Z-score) and comparison of different handling strategies.

In [103…

```
# Create a dataset with normal distribution
data = pd.DataFrame(np.random.standard_normal((1000, 4)))

data.describe()
```

Out[103…

	0	1	2	3
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	0.040816	-0.021516	0.070822	0.066644
std	0.927801	0.986116	1.001320	0.974012
min	-3.080781	-4.159665	-2.870496	-3.714685
25%	-0.601272	-0.668206	-0.564654	-0.565725
50%	0.051558	-0.035718	0.070838	0.088427
75%	0.680546	0.654836	0.718246	0.672199
max	3.189587	3.183569	3.366716	3.380770

Detection: Let's find values exceeding 3 in absolute value (Standard Deviation > 3).

In [104…

```
col = data[2]

# Boolean indexing to find rows where absolute value > 3
col[col.abs() > 3]
```

Out[104…

```
109    3.283287
367    3.366716
746    3.044005
Name: 2, dtype: float64
```

To find **any row** that has an outlier in **any column**, we use `.any(axis=1)`.

In [105…

```
# 1. data.abs() > 3 returns a boolean DataFrame
# 2. .any(axis="columns") checks if any value in the row is True
data[(data.abs() > 3).any(axis="columns")]
```

Out[105…

	0	1	2	3
7	3.189587	1.336925	-1.094954	0.395709
109	0.831208	-1.180172	3.283287	1.172005
250	-0.394586	3.183569	0.729299	-0.251058
367	-2.174851	0.031215	3.366716	-0.385277
507	-1.025720	-1.509659	0.604684	3.380770
559	0.157889	-4.159665	-0.088201	-0.308172
612	-3.080781	-1.046867	0.952873	-1.157002
740	-1.588759	-0.484478	-0.465990	-3.714685
746	-1.181062	0.851212	3.044005	-0.348325

Capping: Instead of removing outliers, we can cap them at a threshold.

In [106…

```
# Set values > 3 to 3, and < -3 to -3, preserving the sign
data[data.abs() > 3] = np.sign(data) * 3
```

In [107…

```
data.describe()
```

Out[107…

	0	1	2	3
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	0.040707	-0.020540	0.070128	0.066978
std	0.926907	0.981335	0.999177	0.970268
min	-3.000000	-3.000000	-2.870496	-3.000000
25%	-0.601272	-0.668206	-0.564654	-0.565725
50%	0.051558	-0.035718	0.070838	0.088427
75%	0.680546	0.654836	0.718246	0.672199
max	3.000000	3.000000	3.000000	3.000000

Removal: Or we can just drop the rows with outliers.

In [108…

```
# Keep rows where ALL columns are within the threshold ( < 3)
data[(data.abs() < 3).all(axis="columns")]
```

Out[108…

	0	1	2	3
0	-1.371479	0.972399	-0.242090	-0.457371
1	0.011831	1.046397	-0.950964	0.680962
2	0.095880	0.228233	-0.772845	0.188823
3	0.788734	0.086496	-1.545757	-0.628817
4	-0.840753	0.576621	0.243015	-0.632210
...	...	...	...	...
995	0.394070	0.457530	0.682690	-0.586604
996	0.498993	0.123070	-1.873255	-0.659084
997	0.266486	0.303699	0.339912	1.367670
998	-0.656245	0.691537	0.179641	0.932063
999	-1.360871	1.371131	0.769870	1.380772

991 rows × 4 columns

Section 3: Refinement & Transformation

Goal: Now that the data is clean, we need to transform it into the right format for analysis.

We will cover:

1. **Mapping & Replacing:** Changing values based on a dictionary logic.
2. **String Manipulation:** Cleaning text data using the `.str` accessor.
3. **Exporting:** Saving your hard work to a file.

Part 1: Transforming Data (Mapping)

Sometimes we need to add new columns based on existing ones. `map()` is perfect for this.

In [109…

```
data = pd.DataFrame({"food": ["bacon", "pulled pork", "bacon", "pastrami",
                              "corned beef", "bacon", "pastrami", "honey ham",
                              "nova lox"],
                     "ounces": [4, 3, 12, 6, 7.5, 8, 3, 5, 6]})

data
```

Out[109…

	food	ounces
0	bacon	4.0
1	pulled pork	3.0
2	bacon	12.0
3	pastrami	6.0
4	corned beef	7.5
5	bacon	8.0
6	pastrami	3.0
7	honey ham	5.0
8	nova lox	6.0

Scenario: We want to add a column indicating the animal source of each food.

In [113…

```
meat_to_animal = {
    "bacon": "pig, pig2",
    "pulled pork": "pig",
    "pastrami": "cow",
    "corned beef": "cow",
    "honey ham": "pig",
    "nova lox": "salmon"
}
```

In [114…

```
# .map() looks up the value in the 'food' column in our dictionary
data["animal"] = data["food"].map(meat_to_animal)

data
```

Out[114…

	food	ounces	animal
0	bacon	4.0	pig, pig2
1	pulled pork	3.0	pig
2	bacon	12.0	pig, pig2
3	pastrami	6.0	cow
4	corned beef	7.5	cow
5	bacon	8.0	pig, pig2
6	pastrami	3.0	cow
7	honey ham	5.0	pig
8	nova lox	6.0	salmon

You can also pass a function to `map()` for custom logic.

In [112…

```python
def get_animal(x):
    return meat_to_animal[x]

data["food"].map(get_animal)
```

Out[112…

```
0       pig
1       pig
2       pig
3       cow
4       cow
5       pig
6       cow
7       pig
8    salmon
Name: food, dtype: object
```

Replacing Values: `replace` is a specialized version of map, great for fixing sentinel values (like -999 for missing data).

In [115…

```python
data = pd.Series([1, -999, 2, -999, -1000, 3])

data
```

Out[115…

```
0       1
1    -999
2       2
3    -999
4   -1000
5       3
dtype: int64
```

In [116…

```python
# Replace -999 with NaN
data.replace(-999, np.nan)
```

Out[116…

```
0       1.0
1       NaN
2       2.0
3       NaN
4   -1000.0
5       3.0
dtype: float64
```

In [117…

```python
# Replace multiple values at once
data.replace([-999, -1000], np.nan)
```

Out[117…

```
0    1.0
1    NaN
2    2.0
3    NaN
4    NaN
5    3.0
dtype: float64
```

In [118…

```python
# Replace with different values (-999 -> NaN, -1000 -> 0)
data.replace([-999, -1000], [np.nan, 0])
```

Out[118…

```
0    1.0
1    NaN
2    2.0
3    NaN
4    0.0
5    3.0
dtype: float64
```

In [119…

```
# Using a dictionary for clarity
data.replace({-999: np.nan, -1000: 0})
```

Out[119…

```
0    1.0
1    NaN
2    2.0
3    NaN
4    0.0
5    3.0
dtype: float64
```

🛠️ Exercise: Transforming Data

Try replacing values in a DataFrame.

In [120…

```
df = pd.DataFrame(np.random.standard_normal((6, 3)))

df.iloc[2:, 1] = -999
df.iloc[4:, 2] = 999

df
```

Out[120…

	0	1	2
0	-0.668418	-0.067381	-2.147094
1	-0.948121	-1.053492	-1.679677
2	0.001960	-999.000000	0.862315
3	1.380691	-999.000000	0.234941
4	0.049871	-999.000000	999.000000
5	0.660035	-999.000000	999.000000

> **Task:** Replace -999 with NaN and 999 with 0.

In [121…

```
df.replace({-999: np.nan, 999: 0})
```

Out[121…

	0	1	2
0	-0.668418	-0.067381	-2.147094
1	-0.948121	-1.053492	-1.679677
2	0.001960	NaN	0.862315
3	1.380691	NaN	0.234941
4	0.049871	NaN	0.000000
5	0.660035	NaN	0.000000

Part 2: String Manipulation

Pandas has a special accessor `.str` that unlocks string methods for an entire Series at once. This handles missing values gracefully.

In [122…

```
data = {"Dave": "dave@google.com", "Steve": "steve@gmail.com",
        "Rob": "rob@gmail.com", "Wes": np.nan}
```

```
data = pd.Series(data)

data
```

Out[122…

```
Dave     dave@google.com
Steve    steve@gmail.com
Rob        rob@gmail.com
Wes                  NaN
dtype: object
```

In [123…

```
# Check if 'gmail' exists in each string
data.str.contains("gmail")
```

Out[123…

```
Dave     False
Steve     True
Rob       True
Wes        NaN
dtype: object
```

Note on Data Types: Pandas has a specialized `StringDType` (`string`) vs the generic `object` type.

In [124…

```
data_as_string = data.astype('string')

data_as_string
```

Out[124…

```
Dave     dave@google.com
Steve    steve@gmail.com
Rob        rob@gmail.com
Wes                 <NA>
dtype: string
```

In [125…

```
data_as_string.str.contains("gmail")
```

Out[125…

```
Dave     False
Steve     True
Rob       True
Wes       <NA>
dtype: boolean
```

Slicing: We can treat the column like a Python string.

In [126…

```
# Get the first 5 characters
data_as_string.str[:5]
```

Out[126…

```
Dave     dave@
Steve    steve
Rob      rob@g
Wes       <NA>
dtype: string
```

Regex: Regular expressions allow for complex pattern matching.

In [127…

```
import re
```

In [128…

```
# Pattern to identify email parts: (username) @ (domain) . (suffix)
pattern = r"([A-Z0-9._%+-]+)@([A-Z0-9.-]+)\.([A-Z]{2,4})"

data.str.findall(pattern, flags=re.IGNORECASE)
```

Out[128…

```
Dave     [(dave, google, com)]
Steve    [(steve, gmail, com)]
Rob        [(rob, gmail, com)]
Wes                        NaN
dtype: object
```

Retrieving elements: We can chain `.str` calls to get specific parts of the regex match.

In [129…

```
matches = data.str.findall(pattern, flags=re.IGNORECASE).str[0]

matches
```

Out[129…

```
Dave     (dave, google, com)
Steve    (steve, gmail, com)
Rob        (rob, gmail, com)
Wes                      NaN
dtype: object
```

In [130…

```
# Get index 1 of the tuple (the domain name)
matches.str.get(1)
```

Out[130…

```
Dave     google
Steve     gmail
Rob       gmail
Wes         NaN
dtype: object
```

The `extract` method is very powerful—it creates a new DataFrame with columns for each captured regex group.

In [131…

```
data.str.extract(pattern, flags=re.IGNORECASE)
```

Out[131…

	0	1	2
Dave	dave	google	com
Steve	steve	gmail	com
Rob	rob	gmail	com
Wes	NaN	NaN	NaN

🛠️ Exercise: Strings

> 1. Get the 1st group of the regex email pattern (the username).
> 2. Convert data into titlecase (e.g. "Dave@Google.Com").

In [133…

```
matches.str.get(0) # getter
# matches.str[0] # indexing
```

Out[133…

```
Dave      dave
Steve    steve
Rob        rob
Wes        NaN
dtype: object
```

In [134…

```
data.str.title()
```

Out[134…

```
Dave     Dave@Google.Com
Steve    Steve@Gmail.Com
Rob        Rob@Gmail.Com
Wes                  NaN
dtype: object
```

Part 3: Writing Data (Exporting)

Once your data is cleaned, you need to save it. `to_csv` is the most common method.

In [135…

```
result.to_csv("../data/out.csv")
```

In [136…

```
!cat ../data/out.csv
```

```
,something,a,b,c,d,message
0,one,1,2,3.0,4,
1,two,5,6,,8,world
2,three,9,10,11.0,12,
```

Tip: You usually want `index=False` to avoid saving the row numbers as a separate column.

In [137…

```
result.to_csv("../data/out.csv", index=False)
```

In [138…

```
!cat ../data/out.csv
```

```
something,a,b,c,d,message
one,1,2,3.0,4,
two,5,6,,8,world
three,9,10,11.0,12,
```

Excel Export:

In [139…

```
# Option 1: Using ExcelWriter (good for multiple sheets)
writer = pd.ExcelWriter('../data/out.xlsx')

frame.to_excel(writer, 'Sheet1')

writer.close()
```

In [140…

```
# Option 2: Direct export
frame.to_excel('../data/out.xlsx')
```

Optional Topics

If time permits, explore these advanced transformations.

Renaming Axis Indexes

Changing row/column labels using mapping.

In [ ]:

```
data = pd.DataFrame(np.arange(12).reshape((3, 4)),
                    index=['Ohio', 'Colorado', 'New York'],
                    columns=['one', 'two', 'three', 'four'])
```

In [ ]:

```
data
```

Using `.map()` on the index:

In [ ]:
```
def transform(x):
    return x[:4].upper()

data.index.map(transform)
```

In [ ]:
```
# Assign back to modify in-place
data.index = data.index.map(transform)
```

In [ ]:
```
data
```

Using `.rename()` (returns a copy by default):

In [ ]:
```
data.rename(index=str.title, columns=str.upper)
```

In [ ]:
```
data.rename(index={"OHIO": "INDIANA"}, columns={"three": 3})
```

Permutation and Random Sampling

Reordering or selecting random subsets.

In [ ]:
```
df = pd.DataFrame(np.arange(5 * 7).reshape((5, 7)))

df
```

In [ ]:
```
# Create a random order of indices
sampler = np.random.permutation(5)

sampler
```

In [ ]:
```
# Reorder rows
df.iloc[sampler]
```

In [ ]:
```
# Alternative: use .take()
df.take(sampler)
```

Permuting columns:

In [ ]:
```
column_sampler = np.random.permutation(df.shape[1])

column_sampler
```

In [ ]:
```
df.take(column_sampler, axis=1)
```

Random Sample: Getting a random subset without manually creating a sampler.

In [ ]:
```
df.sample(n=3)
```

In [ ]:
```
choices = pd.Series([5, 7, -1, 6, 4])

# Sample with replacement (items can be picked more than once)
choices.sample(n=10, replace=True)
```

> **Task:** Sample `df` using the parameter `frac` (percentage) instead of `n` (count).

Categorical Data

Converting string columns to `category` dtype saves memory and speeds up operations.

> 📊 **Visual Illustration Available**: See ![categorical_binning](../visualisations/05_categorical_binning.png `for visualizations comparing` pd.cut() `vs` pd.qcut()`, categorical operations, and dummy variables.

In [ ]:
```
values = pd.Series(["apple", "orange", "apple", "apple"] * 2)

values
```

In [ ]:
```
values.unique()
```

In [ ]:
```
values.value_counts()
```

Under the hood: Categoricals are stored as integers referencing a dictionary of values.

In [ ]:
```
values = pd.Series([0, 1, 0, 0] * 2)

dim = pd.Series(["apple", "orange"])

values
```

In [ ]:
```
dim
```

In [ ]:
```
dim.take(values)
```

Using Pandas `category` type:

In [ ]:
```
fruits = ["apple", "orange", "apple", "apple"] * 2
N = len(fruits)

# to ensure reproducibility
rng = np.random.default_rng(seed=12345)

df = pd.DataFrame({"fruit": fruits,
                   "basket_id": np.arange(N),
                   "count": rng.integers(3, 15, size=N),
                   "weight": rng.uniform(0, 4, size=N)},
                  columns=["basket_id", "fruit", "count", "weight"])

df
```

In [ ]:
```
# Convert object column to category
fruit_cat = df['fruit'].astype('category')

fruit_cat
```

The values for `fruit_cat` are now an instance of `pandas.Categorical`, which you can access via the `.array` attribute:

In [ ]:
```
c = fruit_cat.array

type(c)
```

In [ ]:
```
c.categories
```

In [ ]:
```
c.codes
```

In [ ]:
```
dict(enumerate(c.categories))
```

In [ ]:
```
# Assign back to the dataframe
df['fruit'] = df['fruit'].astype('category')

df
```

Creating from Codes:

In [ ]:
```
categories = ['foo', 'bar', 'baz']
codes = [0, 1, 2, 0, 0, 1]

my_cats = pd.Categorical.from_codes(codes, categories)

my_cats
```

Ordered Categoricals: Useful for Likert scales or sizes (Small < Medium < Large).

In [ ]:
```
ordered_cats = pd.Categorical.from_codes(codes, categories, ordered=True)

ordered_cats
```

Binning Data (`pd.cut`): Converting continuous data (e.g., Age) into categorical bins (e.g., Age Groups).

In [ ]:
```
ages = [20, 22, 25, 27, 21, 23, 37, 31, 61, 45, 41, 32]
```

In [ ]:
```
# Define bin edges
bins = [18, 25, 35, 60, 100]

# Cut the data
age_cat = pd.cut(ages, bins)

age_cat
```

In [ ]:
```
age_cat.codes
```

In [ ]:
```
age_cat.categories
```

In [ ]:
```
age_cat.categories[0]
```

In [ ]:
```
pd.value_counts(age_cat)
```

Binning Parameters:

In [ ]:
```
# right=False makes the left side closed [inclusive, exclusive)
pd.cut(ages, bins, right=False)
```

In [ ]:
```
# Adding custom labels
group_names = ["Youth", "YoungAdult", "MiddleAged", "Senior"]

pd.cut(ages, bins, labels=group_names)
```

If you pass an integer number of bins to `pd.cut`, it will compute equal-length bins based on the min/max of the data.

In [ ]:
```
data = np.random.uniform(size=20)

pd.cut(data, 4, precision=2)
```

> **Task:** Cut `ages` into 5 bins instead of 4. Set the labels to `['Youth', 'YoungAdult', 'MiddleAged', 'Senior', 'Elderly']`.

Categorical Methods (.cat):

In [ ]:
```
s = pd.Series(['a', 'b', 'c', 'd'] * 2)
cat_s = s.astype('category')

cat_s
```

In [ ]:
```
cat_s.cat.codes
```

In [ ]:
```
cat_s.cat.categories
```

Modifying categories:

In [ ]:
```
actual_categories = ['a', 'b', 'c', 'd', 'e']
cat_s2 = cat_s.cat.set_categories(actual_categories)

cat_s2
```

In [ ]:
```
cat_s.value_counts()
```

In [ ]:
```
cat_s2.value_counts()
```

In [ ]:
```
cat_s2.cat.remove_unused_categories()
```

Computing Indicator / Dummy Variables: Converting categorical variables into binary columns (One-Hot Encoding) for machine learning.

In [ ]:
```
df = pd.DataFrame({"key": ["b", "b", "a", "c", "a", "b"],
                   "data1": range(6)})

df
```

In [ ]:
```
pd.get_dummies(df["key"])
```

In [ ]:
```
dummies = pd.get_dummies(df["key"], prefix="key")

dummies
```

Recipe: Combining `get_dummies` with `cut`.

In [ ]:
```
np.random.seed(12345)

values = np.random.uniform(size=10)

values
```

In [ ]:
```
bins = [0, 0.2, 0.4, 0.6, 0.8, 1]

pd.get_dummies(pd.cut(values, bins))
```

> **Task:** Cut values into 4 bins and create dummy variables.

Databases

Connecting to a database using `sqlalchemy`.

In [ ]:
```
import sqlalchemy as sqla
```

In [ ]:
```
import os

parent_dir = os.path.abspath(os.path.pardir)
```

In [ ]:
```
# Create connection string
engine = sqla.create_engine(f'duckdb:///{parent_dir}/data/unit-1-4.db')
```

In [ ]:
```
# Read entire table
df = pd.read_sql('resale_flat_prices_2017', engine)
```

In [ ]:
```
df
```

In [ ]:
```
# Execute raw SQL query
df = pd.read_sql("SELECT * FROM resale_flat_prices_2017", engine)
```

In [ ]:
```
engine.table_names()
```

Binary files (Pickle)

`pickle` is Python's native serialization format. Good for short-term storage, but not for sharing data between different languages.

In [ ]:
```
result.to_pickle('../data/out.pkl')
```

In [ ]:
```
data = pd.read_pickle('../data/out.pkl')

data
```

> **Task:** Write a filtered DataFrame to a new database table.
>
> 1. Filter for flats in "YISHUN".
> 2. Write to table `yishun_flat_prices_2017`.

In [ ]:
```
df_yishun = df[df.town == "YISHUN"]

df_yishun.to_sql("yishun_flat_prices_2017", engine)
```

> **Final Challenge:**
>
> 1. Read only flats from `BISHAN` to a new dataframe.
> 2. Write the dataframe to a new database table `bishan_flat_prices_2017`.